

Day 6 — NumPy & Data Formats

Arrays, Vectorized Operations & File Format Tradeoffs — Notes for Recap

1. Arrays

A NumPy array is similar to a Python list, but stores data more efficiently and enables fast, whole-array mathematical operations. It is the underlying structure that pandas itself is built on.

```
import numpy as np

prices = np.array([100, 250, 75, 300])
print(prices)          # [100 250  75 300]
print(prices.shape)    # (4,)
print(prices.dtype)    # int64
```

Key takeaway: *Arrays are the foundation pandas DataFrames are built on top of - understanding arrays makes it easier to understand why pandas behaves the way it does.*

2. Vectorized Operations

Vectorization means applying an operation to an entire array at once, instead of looping through each element manually. This is both far shorter to write and significantly faster to run.

```
prices = np.array([100, 250, 75, 300])

# Vectorized - fast, one line
discounted = prices * 0.9

# The slow, manual way (avoid this)
discounted_manual = []
for p in prices:
    discounted_manual.append(p * 0.9)

print(discounted)    # [ 90. 225.  67.5 270. ]
```

Key takeaway: *Whenever you find yourself writing a for loop to do math on every item in a list, that's usually a sign a vectorized NumPy operation would be faster and cleaner.*

3. Numerical Ops

NumPy provides built-in functions for common numerical calculations - sums, averages, min/max, and standard deviation - all computed efficiently across an entire array.

```
prices = np.array([100, 250, 75, 300])

print(np.sum(prices))      # 725
print(np.mean(prices))    # 181.25
print(np.max(prices))     # 300
print(np.min(prices))     # 75
print(np.std(prices))     # standard deviation
```

Key takeaway: These same operations exist in pandas too (built on top of NumPy) - learning them here means the pandas versions will already feel familiar.

4. CSV / JSON / Parquet

These are the three file formats you'll encounter constantly in data engineering, each with different tradeoffs.

Format	Characteristics
CSV	Plain text, human-readable, no data types preserved, larger file size, universally supported
JSON	Text-based, supports nested/structured data, larger file size, common for APIs
Parquet	Binary, columnar, compressed, preserves data types, much faster to read for analytics

```
import pandas as pd

df = pd.read_csv("sales.csv")

df.to_json("sales.json")
df.to_parquet("sales.parquet")

# Reading back
df_parquet = pd.read_parquet("sales.parquet")
```

Key takeaway: CSV is easiest to inspect by eye, JSON is best for nested/API data, and Parquet is what production data pipelines use for speed and storage efficiency at scale. Knowing when to use each is a core data engineering decision.

5. API Responses

APIs almost always return data as JSON. Turning that JSON response directly into a pandas DataFrame is one of the most common first steps in a data pipeline.

```
import requests
import pandas as pd

response = requests.get("https://api.example.com/orders")
data = response.json()          # converts the response into a Python dict/list

df = pd.DataFrame(data)        # turns it into a DataFrame
print(df.head())
```

Key takeaway: Always check the structure of an API response before assuming it will convert cleanly into a flat table - nested JSON (lists inside dictionaries inside dictionaries) often needs extra flattening before `pd.DataFrame()` works as expected.

Day 6 Summary — NumPy & Data Formats

Concept	One-Line Recap
Arrays	Efficient, fixed-type data structure that pandas is built on top of
Vectorized Operations	Applying math to a whole array at once instead of looping
Numerical Ops	Built-in sum/mean/min/max/std functions across an array
CSV / JSON / Parquet	Three common file formats, each with different speed/size/structure tradeoffs
API Responses	Converting JSON from an API directly into a usable DataFrame

Overall takeaway: Day 6 connects the dots between raw numerical performance (NumPy) and how data actually moves between systems (file formats and APIs) - both are foundational to everything that comes later in the roadmap, especially when performance and file size start to matter at scale.